Master 2 Internship report

Dynamical properties of the Hopfield model through bitwise arithmetic

Bastien Dumont

Supervised by Michele Castellana and David Lacoste

Institut Curie and ESPCI, Paris

March 23 - July 17, 2026

Master de Physique fondamentale et applications, parcours Physique

Scholar year 2025 - 2026

July 3, 2026

placeholder

Abstract

placeholder

Contents

1 Introduction 5
- 1.1 Presentation of the host laboratories . . . 5
- 1.2 Subject . . . 5
- 1.3 Objectives . . . 5

2 Implementation of the algorithm 6
- 2.1 The Metropolis algorithm . . . 6
- 2.2 First optimization: the bitwise arithmetic . . . 6
- 2.3 Second Optimization: Shared random numbers . . . 7
- 2.4 Note on the independence of the realizations . . . 8
- 2.5 Code structure . . . 8
- 2.6 Standard implementation of the Metropolis algorithm . . . 9

3 The Ising model 11
- 3.1 The model . . . 11
- 3.2 Bitwise Implementation . . . 11
- 3.3 Phase diagram . . . 12
- 3.4 Performance test . . . 13

4 The Spinglass model 16
- 4.1 The Edwards–Anderson model . . . 16
- 4.2 Bitwise Implementation . . . 16
- 4.3 Performance test . . . 16

5 The Hopfield model 18
- 5.1 The model . . . 18
- 5.2 Flip condition . . . 20
- 5.3 Bitwise Implementation . . . 20
- 5.4 Capacity of the Hopfield network . . . 22
- 5.5 Performance test . . . 22

6 Conclusion 25

Appendices 26
- A Naive implementation of the classic Metropolis algorithm . . . 26
- B Bitwise implementation of the Metropolis algorithm for the spin glass model . . . 26

1 Introduction

1.1 Presentation of the host laboratories

This internship was conducted as part of my Master 2 at the Magistère of Fundamental Physics of Paris-Saclay Université, which I completed at Aix-Marseille Université in the "Physique Fondamentale et Applications" track. It was carried out at the "Physique des Cellules et Cancer" (PCC, UMR 168) laboratory of the Institut Curie as well as at the Gulliver laboratory (UMR 7083) of the "École Supérieure de Physique et de Chimie Industrielles" (ESPCI) in Paris, and was supervised by Michele Castellana (Institut Curie) and David Lacoste (ESPCI). Although the PCC unit is a leading biological physics laboratory, part of the Institut Curie – a pioneer institution in cancer research and treatment – it hosts a renowned theory team (of which Michele Castellana is a member) that is sometimes interested in physics-driven topics such as the one investigated in this report. The Gulliver laboratory hosts both experimentalists and theoreticians (among whom David Lacoste), working at the frontier between physical, chemical and biological topics.

1.2 Subject

In October 2024, John J. Hopfield was awarded the Nobel Prize in Physics alongside Geoffrey Hinton for their pioneering work on artificial neural networks [1]. Their work laid the foundations for further development and understanding of neural networks, whether they are organic or artificial, and eventually gave rise to Artificial Intelligence (AI).
Hopfield worked on an artificial neural network, originally invented by Shun'ichi Amari in 1972 [2]. He greatly contributed to the analysis, understanding and popularization of the network in his landmark paper *Neural networks and physical systems with emergent collective computational abilities* published in 1982 [3], and the network was eventually named after him. The model, described in more details in subsection 5.1, is similar to the Ising and the Spin Glass networks: it is composed of a set of spins – called neurons in the case of the Hopfield network – $S_i \in \{-1, 1\}$ interacting with one another. In the same way as the two preceding models, the Hopfield model is energy-based, that is it will tend to minimize its energy over time. What makes this model different from the other two is the way the interactions between neurons are defined: as for the Ising and spin glass models they remain static over time but depend on the set of patterns to be stored in the network, making the latter attractors, that is to say minima of energy, of the model.
The Hopfield model provided the first simple yet powerful framework for understanding networks of interconnected neurons. It has become a reference in statistical physics on networks and its properties at equilibrium have been extensively studied and are now well known, whether it is its statistical mechanics [4–7] or its phase diagram and transitions [8]. However, much less is known about the dynamics that lead to those well-known equilibrium properties.

1.3 Objectives

The main objective of this internship is to efficiently simulate the Metropolis dynamics on the Hopfield network. To this end, we use a bitwise formalism that allows us to simulate several replicas of the system in parallel, increasing the efficiency of the exploration of the model's energy landscape. Our objectives in this internship are thus twofold: first, to validate the bitwise implementation and quantify its computational gain in yield over the standard Metropolis algorithm; second, to leverage this implementation to study the dynamics of the Hopfield network. To validate the implementation, we begin with the Ising model, which serves as a natural benchmark due to its simplicity and well-characterized phase transition. Once this implementation has been proven sane and efficient on the Ising network, we increase the complexity by implementing it on the spin glass network, testing the gain in yield again.
Once the bitwise arithmetic has been validated on both models, we implement it on the Hopfield network in order to analyze its dynamics.

2 Implementation of the algorithm

We start by presenting the broad implementation of the Metropolis algorithm, before showing how we adapt it to each specific model.

2.1 The Metropolis algorithm

In order to make the models evolve in time, which is necessary to observe their dynamics, we use the well-known metropolis algorithm [9]. For models with a defined Hamiltonian, the Metropolis algorithm proceeds as follows:

1. Start from an initial configuration $\{S_i\}$.
2. Repeat N times:
   a. Pick a random spin S_k and compute the energy change ΔE_k upon flipping it.
   b. If $\Delta E_k \leq 0$, accept the flip.
   c. Else, draw a random number $r \sim \mathcal{U}([0,1])$ and accept the flip if $r \leq e^{-\beta \Delta E_k}$.
3. Repeat Step 2 for N_{sweeps} sweeps.

Where N is the number of spins, β is the inverse temperature, and N_{sweeps} is the number of sweeps, where one sweep consists of N successive single-spin flip attempts.
This algorithm however becomes computationally demanding when one attempts to sample a large number of configurations. This situation occurs, for example, when looking for rare samples in the dynamics, or sampling many different initial conditions – both of which we will try to explore during this internship. We thus need to find way of optimizing the implementation of the algorithm.

2.2 First optimization: the bitwise arithmetic

The first optimization in the implementation of the Metropolis algorithm is the use of bitwise arithmetic. The main idea is to leverage the boolean representation as stored in our computers. The typical object that we manipulate is called `Bits`: it is a collection of $n_{\text{bits}} = 64$ boolean variables. In that way, we have the possibility to simulate 64 evolutions of the same variable in parallel.

$$W = \underbrace{\boxed{b^0}\boxed{b^1}\boxed{b^2}\cdots\boxed{b^{62}}\boxed{b^{63}}}_{64\text{ bits} \leftrightarrow 64\text{ replicas}}$$

Figure 1: A `Bits` W encoding 64 parallel replicas. Each bit b_i carries the state of replica i.

This structure is very convenient to represent binary variable such as the spins of the system. The boolean representation $b_i \in \{0,1\}$ of a spin $s_i \in \{-1,+1\}$ is given by $b_i = (s_i+1)/2$. In that way, we are able to implement several replicas of the full spin system in a set of `Bits` $\{S_i\}$, each encoding the replicas of the associated spin, as shown in Figure 2.

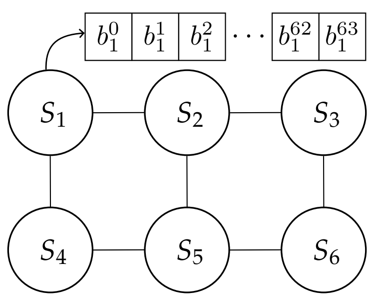


Figure 2: Bitwise implementation of a spin system. Each replica of the system is obtained by selecting the corresponding replica in each spin.

Storing all replicas in a single `Bits` object allows us to manipulate and update them simultaneously using a single bitwise operator. This is an instance of SWAR (SIMD Within A Register) parallelism, where SIMD stands for Single Instruction, Multiple Data: all 64 replicas are updated by a single bitwise instruction. The restriction to $n_{\text{bits}} = 64$ reflects the native word size of modern 64-bit CPUs [10]. This could in principle be extended to 512 bits using AVX-512 SIMD instructions, at the cost of introducing a new register type in the implementation. This will be left for further discussion.
The convention used for what follows is presented in the Table 1:

Object	Notation
Scalar spin	$s_i \in \{-1, +1\}$
Scalar bit	$b_i \in \{0, 1\}$
64 replicas of spin i	$S_i \in \{-1, 1\}^{64}$
64-bit word (`Bits`)	$B_i \in \{0, 1\}^{64}$
Replica k of spin i	$s_i^{(k)} \in \{-1, +1\}$
Replica k of bit i	$b_i^{(k)} \in \{0, 1\}$

Table 1: Notation for the bitwise representation of spins.

Now that we know how to conveniently handle binary variables such as spins, we need to employ the same formalism to represent integer, such as the sum ΔE_k. This is done using a `BitSet`, which is a collection of `Bits`. When each `Bits` is associated with a power of 2, the `BitSet` represent a collection of n_{bits} unsigned integers and the `BitSet` is called `UnsignedInt`. We show an example of `UnsignedInt` in the Figure 3.

$$Z = \begin{array}{c}
\boxed{b_0^0 \mid b_0^1 \mid b_0^2} \cdots \boxed{b_0^{62} \mid b_0^{63}} \quad 2^0 \\
\boxed{b_1^0 \mid b_1^1 \mid b_1^2} \cdots \boxed{b_1^{62} \mid b_1^{63}} \quad 2^1 \\
\vdots \qquad\qquad\qquad\qquad \vdots \\
\underbrace{\boxed{b_l^0 \mid b_l^1 \mid b_l^2} \cdots \boxed{b_l^{62} \mid b_l^{63}}}_{\text{64 replicas}} \quad 2^l
\end{array}$$

Figure 3: An `UnsignedInt` encoding 64 parallel unsigned integers. Each row is a `Bits` associated with a power of 2. The value of each integer is obtained by reading the `UnsignedInt` along the corresponding column. For instance, the fifth integer stored in Z is $Z_4 = \sum_{i=0}^{l} b_i^4 \, 2^i$.

In the same way as for `Bits`, the `UnsignedInt` data structure enables us to add, subtract or even multiply two sets of 64 integers with just the cost of one operation.
This implementation method has proven efficient when dealing with discreet values models such as the Ising [11] or the Spin Glass [12] models, but it has not yet been employed to study the Hopfield network.

2.3 Second Optimization: Shared random numbers

The bitwise implementation is a structural optimization of the data representation. On top of this structural optimization can be added a physical one which consists in having all replicas share the same sequence of random numbers, fully parallelizing their evolution. This turns out to be central: random number generation is often the simulation bottleneck, as it involves costly operations such as multiplications and is called at every step of the hot loop. A schematic view of the shared random number is shown in Figure 4. Using the same random number among all evolutions raises the questions of the independence of the realizations, this will be the subject of the next subsection.

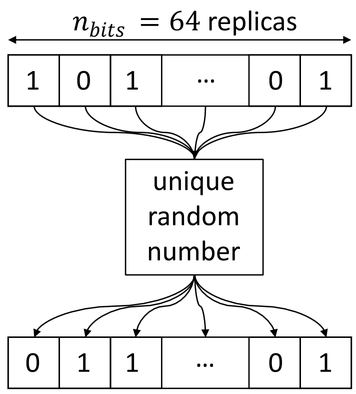


Figure 4: All 64 replicas of the system share the same random number at each step.

2.4 Note on the independence of the realizations

2.5 Code structure

We here present the structure of the code. It has been entirely written in C++, taking advantage of the object-oriented structure to factor out the common parts for each model while enabling them to evolve depending on their specificities. Most of the time of this internship has been spent understanding the object-oriented syntax and philosophy, and implementing an easily readable and efficient code for the simulation of the Metropolis algorithm on the different models. The structure found so far is present in Figure 5. Once the dynamics is simulated, the outputs are analyzed through python scripts.

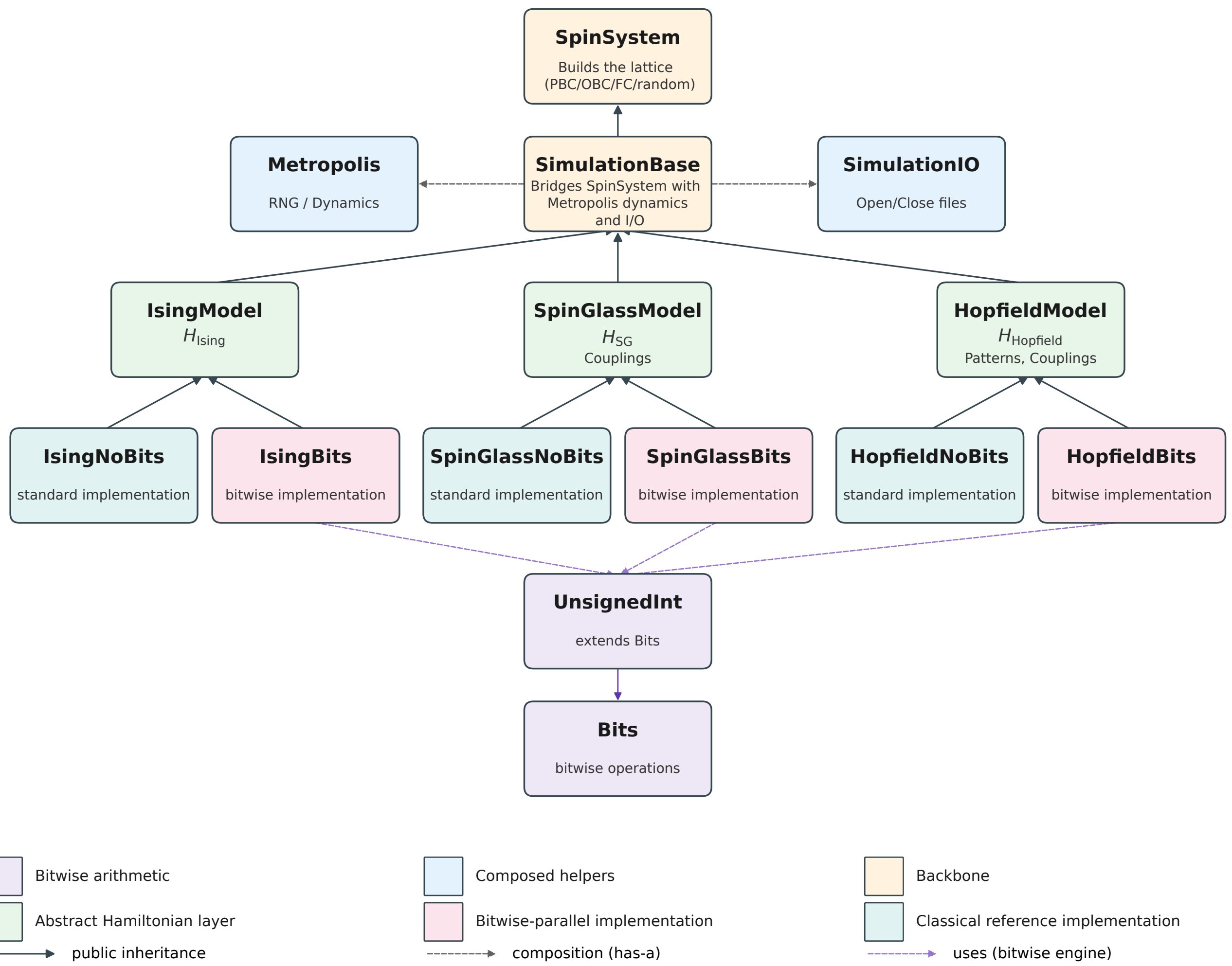


Figure 5: Code structure

The code is built around hierarchical classes. We first implement the topology of the network, which is independent of the type of spin interaction and of the dynamical model used to make the system evolve. On this lattice, we set up the evolution rules (*i.e.* the Metropolis algorithm in our case), and the input/output file management. Then, the combination (topology, dynamics) is applied to one of the three models implemented (Ising, Spin glass or Hopfield) and is implemented either in its standard or bitwise version.

2.6 Standard implementation of the Metropolis algorithm

We show here the standard implementation of the Metropolis, which will be broadly shared across all models.
If $\Delta E_k \leq 0$ the flip is accepted. Else, the flip condition $r \leq e^{-\beta \Delta E_k}$, yields:

$$\begin{aligned} \frac{-1}{\beta} \log r &\geq \Delta E_k \\ \frac{-1}{2\beta J} \log r &\geq S_k \sum_{i \in \partial k} S_i. \end{aligned} \tag{1}$$

With $r \sim \mathcal{U}([0,1])$, we know that $-\log r \sim \mathcal{E}(1)$ [] (source?), thus $-(2\beta J)^{-1} \log r = \rho \sim \mathcal{E}(2\beta J)$. The left-hand side is a random variable of probability density $p(\rho) = 2\beta J e^{-2\beta J \rho}$ and mean $(2\beta J)^{-1}$. The right-hand side of the equation is an integer which can be positive of negative. However, due to the first escape condition of the algorithm, we know that it is strictly positive The left-hand side is thus compared with a positive integer. We can thus only consider the whole part of it without changing the inequality in order to obtain an integer-only flip condition:

$$\lfloor \rho \rfloor \geq S_k \sum_{i \in \partial k} S_i = \Lambda_k. \tag{2}$$

A first naive implementation of the classical algorithm is given in the Algorithm 4 in the appendix, where each of the n_{bits} branch possesses its own random number. A more efficient implementation would be to share the drawn random threshold across the replicas. This shared random number version yields a fast escape condition, taking advantage of the inequality

$$-\max \Lambda_k \leq \Lambda_k \leq \max \Lambda_k, \tag{3}$$

where $\max \Lambda_k$ only depend on the topology of the network. Plugging this into Eq (2) yields the flip escape condition:

$$\lfloor \rho \rfloor \geq \max \Lambda_k \tag{4}$$

When satisfied, this guarantees that Eq. (2) holds for every replica regardless of its local configuration, so that all replicas are flipped simultaneously without computing the per-replica bitwise test. This first escape is useful, as it avoids the bitwise flipping test calculations for each replica, even when done in parallel. Instead, a simple integer comparison suffices. This condition is typically met in the high-temperature regime.
We show in the Algorithm 1 the shared random number implementation of the Metropolis algorithm. This implementation is valid for all models, given a change in the threshold in (4) to be adapted to each

model.

Algorithm 1: Standard Metropolis implementation

Input: Lattice spins $\{S_i\}$, coupling J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

```
for sweep = 1 to N_sweeps do
    for k = 1 to N do
        Draw ⌊ρ⌋ with ρ ~ ℰ(2βJ)
        if ⌊ρ⌋ ≥ max Λ_k then
            for r = 1 to n_bits do
                s_k^(r) ← −s_k^(r)
        else
            for r = 1 to n_bits do
                Compute Λ_k^(r)
                if Λ_k^(r) ≤ 0 or ⌊ρ⌋ ≥ Λ_k^(r) then
                    // the test Λ_k^(r) ≤ 0 is not necessary as ⌊ρ⌋ has already been drawn,
                       but is shown for pedagogy
                    s_k^(r) ← −s_k^(r)
```

This reduces the number of random draws per n_{bits} cycle update to just one, shared among all replicas. This optimization is possible even though the structure is not fully parallelized across replicas. The only case for which this parallelized random number optimization might not be as efficient as expected is when $\Lambda_k^{(r)} \leq 0$ for all replicas simultaneously, in which case a random threshold is drawn but never used.

3 The Ising model

We first implement the method on the Ising model, which is the simplest one.

3.1 The model

The Hamiltonian of the Ising network is given by

$$H_{\text{Ising}} = -J \sum_{\langle i,j \rangle} S_i S_j - h \sum_i S_i,$$

which, in absence of external field h becomes,

$$H_{\text{Ising},\, h=0} = -J \sum_{\langle i,j \rangle} S_i S_j. \tag{5}$$

When flipping the spin S_k ($S_k \longrightarrow -S_k$), the associated energy shift is

$$\begin{aligned} \Delta E_k &= H_{\text{Ising}}\left[-S_k\right] - H_{\text{Ising}}\left[S_k\right] \\ &= 2JS_k \sum_{i \in \partial k} S_i, \end{aligned} \tag{6}$$

where $i \in \partial k$ denotes the summation over the neighbors of k. We now need to implement the flip condition $\lfloor \rho \rfloor \geq \Lambda_k$ in a bitwise formalism in order to properly parallelize the evolutions.

3.2 Bitwise Implementation

Remembering that S_i is actually an array containing 64 replicas of the same spin, we write $S_i = -1 + 2B_i$, with B_i a `Bits`. The sum hence becomes:

$$\begin{aligned} \sum_{i \in \partial k} S_k S_i &= \sum_{i \in \partial k} (-1 + 2B_k)(-1 + 2B_i) \\ &= \sum_{i \in \partial k} 1 - 2(B_k + B_i) + 4B_k.B_i, \end{aligned} \tag{7}$$

We show in Table 2 that the term in the sum can be re-written using the XNOR ($\odot$) logical operator. Thus, the term in the sum is simply equal to $2B_k \odot B_i - 1$, where $\odot$ denotes the bitwise XNOR operation

b_k	b_i	$1 - 2(b_k + b_i) + 4b_k.b_i$	$b_k \odot b_i$	$2b_k \odot b_i - 1$
0	0	1	1	1
0	1	−1	0	−1
1	0	−1	0	−1
1	1	1	1	1

Table 2: The summed term can be simplified using a XNOR operator.

applied to the corresponding bits of each `Bits`. The sum thus becomes:

$$\sum_{i \in \partial k} S_k S_i = 2 \sum_{i \in \partial k} C_{ki} - n_k, \tag{8}$$

with $C_{ki} = 2B_k \odot B_i$ and n_k is the number of neighbor of the spin k. The bitwise flip condition is then:

$$\lfloor \rho \rfloor + n_k \geq 2 \sum_{i \in \partial k} C_{ki}. \tag{9}$$

Moreover, the integer flip condition (4) constrains the range of values taken by $\lfloor \rho \rfloor$ and $2 \sum_{i \in \partial k} C_{ki}$ to n_k and $2n_k$, respectively (with $\max \Lambda_k = n_k$). This is particularly convenient, as it allows the `UnsignedInt` objects to be allocated with static sizes at initialization, removing the need for dynamic resizing during the simulation.

The pseudocode for the bitwise implementation of the Metropolis algorithm on this Ising network is presented in the algorithm 2.

Algorithm 2: Bitwise Ising Metropolis

Input: Bit-replicated spins $\{B_i\}$, coupling J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

```
Variables: Bits xnor, mask
           UnsignedInt T_k, threshold
for sweep = 1 to N_sweep do
    for step = 1 to N do
        Draw k among N
        Draw ⌊ρ⌋ with ρ ~ ℰ(2βJ)
        if ⌊ρ⌋ ≥ n_k then
            B_k ← ¬B_k    // escape condition: unconditional flip across all replicas
        else
            T_k ← 0
            threshold ← 0
            for i ∈ ∂k do
                xnor ← B_k ⊙ B_i                    // bitwise XNOR across replicas
                T_k += xnor               // increment agreement count per replica
            T_k ← 2T_k
            threshold ← ⌊ρ⌋ + n_k
            mask ← (T_k ≤ threshold)                  // per-replica flip condition
            B_k ← B_k ⊕ mask      // bitwise spin flip (XOR operation): only replicas
                                               // for which mask^(ℓ) = 1 are flipped
```

3.3 Phase diagram

We now need to make sure the bitwise implementation is consistent. This is firstly done by checking that it yields exactly the same spin configurations as the standard implementation with shared random numbers when starting from the exact same initial condition and using the same generative seed. Secondly, we can plot the phase diagram of the model and verify that it is agreement with the known analytical results. We show in Figure 6 the obtained phase diagram for several sizes of two-dimensional square lattices. This phase diagram is obtained by successive cooling the system: starting from the highest temperature, we progressively decrease it, using the final state of each step as the initial configuration for the next. This reduces the computational cost compared to reinitializing the network randomly at each temperature point, since the equilibrium state at each temperature lies close to that of the previous one, requiring fewer sweeps to reach thermalization. It aslo prevents the system to get stuck into metastable states such as the strip state for which several (usually two) regions of opposite magnetizations arise when the system is initialized randomly from a low temperature.

Qualitatively, it appears that the magnetization curve get closer to the exact solution when the lattice size is increased. This was to be expected as we gradually get closer to the infinite size case when increasing the lattice size. We also notice an excellent agreement between the exact analytical magnetization and the obtained ones in the ferromagnetic part of the phase diagram. Thus, the obtained phase diagram is in excellent agreement with the expected phase diagram, taking into account that we work on finite size lattices. In this plot, each point corresponds to the mean over the $n_{\text{bits}} = 64$ independent realizations. We show in Figure 7 the trajectories over time of the magnetizations for those 64 realizations.

In order to make sure that the phase transition of our network is well at $T_c = 2.269$ for all lattice sizes,

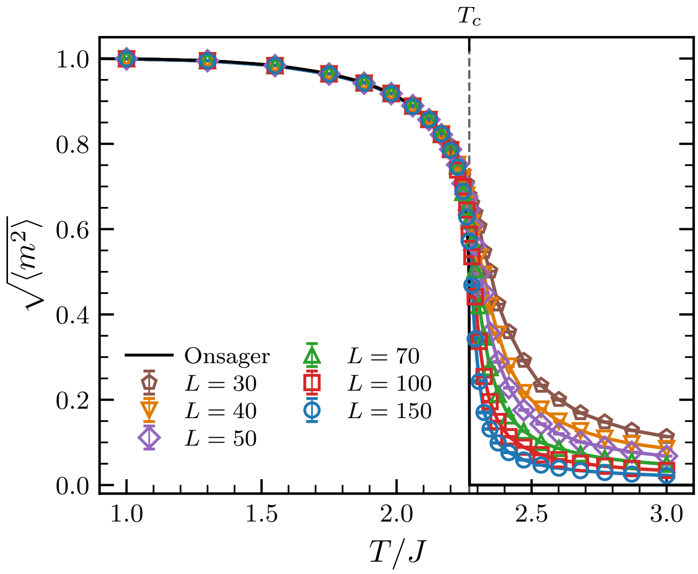


Figure 6: Phase diagram of the Ising model for several sizes of square lattices. The network is built with Periodic Boundary Conditions (PBC). The black line corresponds to the exact analytical solution found by Onsager in the case of an infinite square lattice [13]: $m(T) = 0$ for $T \geq T_c$ and $m(T)_\pm = \pm \left[1 - \sinh^{-4}(2K)\right]^{1/8}$ for $T \leq T_c$, with $K = J/T$ and $T_c = 2J/\log\left(1 + \sqrt{2}\right) \approx 2.269$.

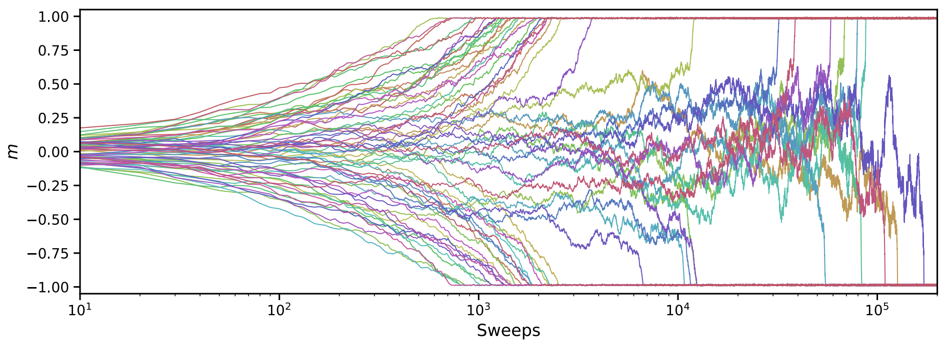


Figure 7: Trajectories of the 64 magnetizations over time for $L = 100$ and $T/J = 1.5$. All those trajectories have been obtained from the same simulation. While some realizations quickly collapse into the equilibrium attractors $m_\pm = \pm 0.953$, some require $\sim 10^2$ times more sweeps to reach the same equilibrium.

we plot the Binder U_L, defined as:

$$U_L = 1 - \frac{\langle m^4 \rangle_L}{3 \langle m^2 \rangle_L^2} \tag{10}$$

Its limit values are $U_L(0) = 2/3$ and $U_L(T \to \infty) = 0$. Most importantly, it has been shown that all Binder curves for different lattice sizes intersect at the same scale-invariant critical point (T_c, U^*) [14]. This is because at T_c, the system becomes scale invariant, making the Binder cumulant independent of the system size at this specific temperature. We show in Figure 8 the plot of the Binder cumulants for the different lattice sizes. We obtain the expected curves for the different lattice sizes, with the known asymptotic values and the crossing of the curves in (T_c, U^*), which validates our implementation.

3.4 Performance test

Now that our algorithm has been shown to produce correct results, we wish to estimate its performance compared to the usual naive implementation. To this end, we measure the ratio of the computational time required to evolve the system over 2^{16} sweeps using both the classical naive

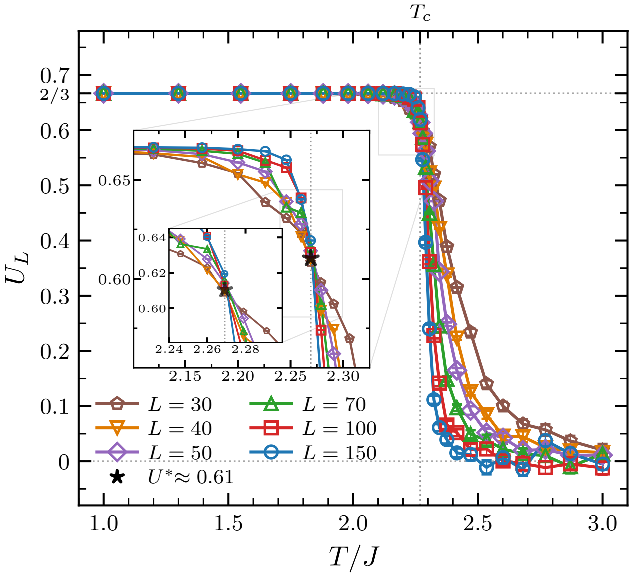
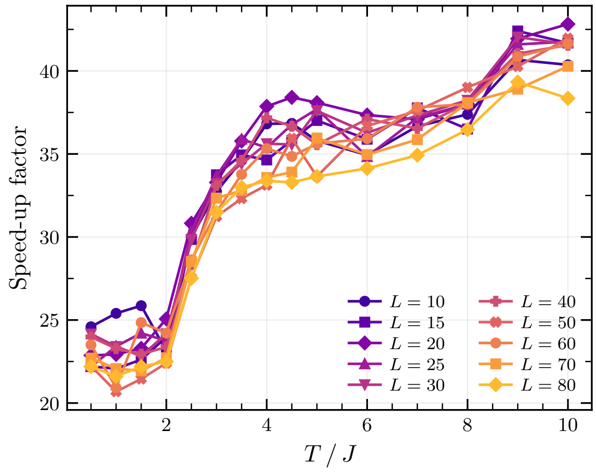


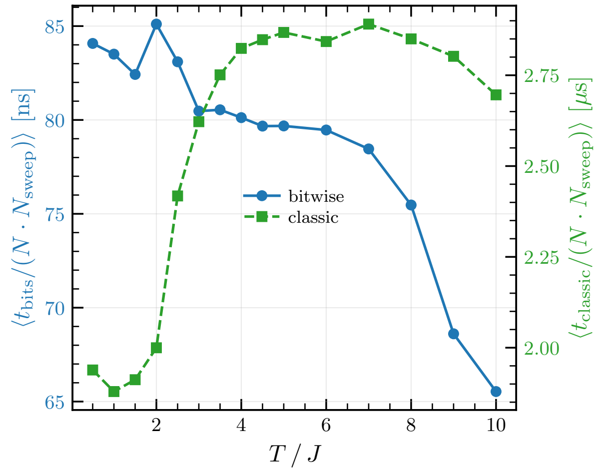
Figure 8: Binder cumulant for the different lattice sizes. The curves approach the two asymptotic limits and intersect at (T_c, U^*), as expected. Insets show the zoomed region around the critical point.

implementation and the bitwise implementation, for various square lattice sizes and temperatures. The result of this performance test is shown in Figure 9. The speedup factor lies between 20 and 45, which is a substantial increase. We notice that this speedup factor consistently increases with the temperature. As shown in (c), the early increase is mostly the effect of the ordering of the configurations at low temperature: before the phase transition, the state collapses into an ordered phase where all spins are aligned. This makes the unconditional flip condition $\Delta E_k \leq 0$ more frequent in the classical implementation, which is computationally cheaper, thus making it faster. This unconditional flip condition however becomes less probable when we increase the temperature, making the normalised time of the classic implementation and the speedup factor increase (especially near the phase transition temperature T_c) up to a certain value where the normalised time stops increasing, probably as the unconditional flip condition is never met anymore. We also notice that the unconditional flip condition in the bitwise implementation ($\lfloor \rho \rfloor \geq n_k$) becomes more and more probable with the increasing temperature, making the bitwise implementation faster. These two effects combined explain the overall shape of the speedup factor evolution, where the first increase is coming from the classical implementation and the second one is the effect of the bitwise. It should be noted that this overall tendency is also expected in the case of a comparison with the standard Metropolis algorithm, as both argument still hold. The increase seems to be uncorrelated with the size of the system. In theory, the speedup factor can even be larger than 64, as the structural optimization alone can potentially yield a speedup of 64 by itself, as it enables us to make 64 realizations evolve in parallel. However, this structural optimization comes with the cost of creating and storing the bitwise objects used to parallelize the evolution, thus reducing the net performance improvement.

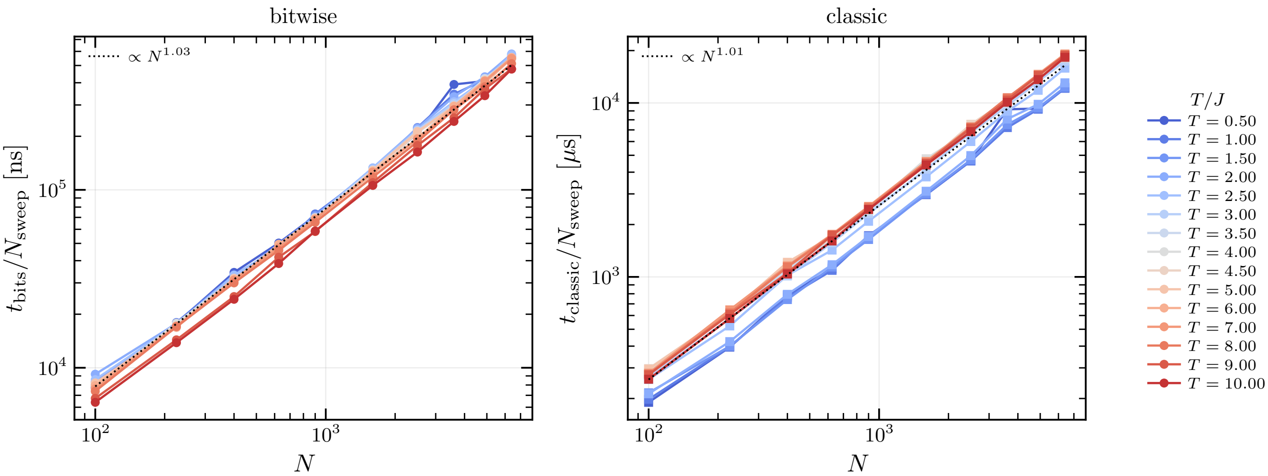
The Subfigure (c) shows that the time required grows proportionally to the system size for both implementation. The complexity of the bitwise implementation thus seems to be $O\left(N \cdot N_{\text{sweeps}}\right)$ while that of the naive classic implementation should be about $O\left(n_{\text{bits}} \cdot N \cdot N_{\text{sweeps}}\right)$. It must be noted that, while we here measure an increase in the speed of the code for a fixed number of outputs, the true advantage of the bitwise implementation is to produce more independent results within the same computational time.



(a) Speedup factor evolution.



(b) Normalized time evolution.



(c) Sweep time depending on the lattice size for all temperatures.

Figure 9: Performance analysis of the bitwise and classical Ising Metropolis implementations. Subfigure (a) shows the evolution of the speedup factor with the temperature for several size of square lattices. (b) shows the evolution of the normalized spin uptade time with the temperature, averaged over the different lattice sizes. Finally, (c) presents the evolution of the time taken to perform one sweep with the lattice size for different temperatures.

4 The Spinglass model

We now move on a more complex model, in order to approach the structure of the Hopfield network.

4.1 The Edwards–Anderson model

The Edwards–Anderson (EA) model – first introduced by Edwards and Anderson in 1975 [15] – is a finite dimensional spin glass model with short range interaction. It is not a mean field spin glass model with infinite range interactions such as the Sherrington–Kirkpatrick model [16]. The model is formally described as follows:

$$H_{\mathrm{EA}} = \sum_{\langle i,j\rangle} J_{ij} S_i S_j. \tag{11}$$

Where J_{ij} is a random variable taking its values in $\{-1, 1\}$ with equal probability. Thus, it can easily be represented by a `Bits`. Under a spin flip ($S_k \to -S_k$), the energy shift is:

$$\Delta E_k = 2S_k \sum_{i\partial k} J_{ki} S_i. \tag{12}$$

The structure being really similar to that of (6), we can perform the same derivation as for the Ising model and find the spin flip condition:

$$\lfloor \rho \rfloor \geq S_k \sum_{i\in\partial k} J_{ki} S_i, \tag{13}$$

with this time $\rho \sim \mathcal{E}(2\beta)$.

4.2 Bitwise Implementation

We now need to express the product $J_{ik} S_k S_i$ in a binary way. As shown in the previous section, the product $S_k S_i$ can be re-written $(-1+2B_k)(-1+2B_i) = -1+2C_{ki}$ with $C_{ki} = B_k \odot B_i$. Thus, by writing $J_{ki} = -1 + 2K_{ki}$ with K_{ki} a `Bits`, we have:

$$\begin{aligned} J_{ik} S_k S_i =& (-1+2K_{ki})(-1+2C_{ki}) \\ =& -1 + 2D_{ki}, \end{aligned} \tag{14}$$

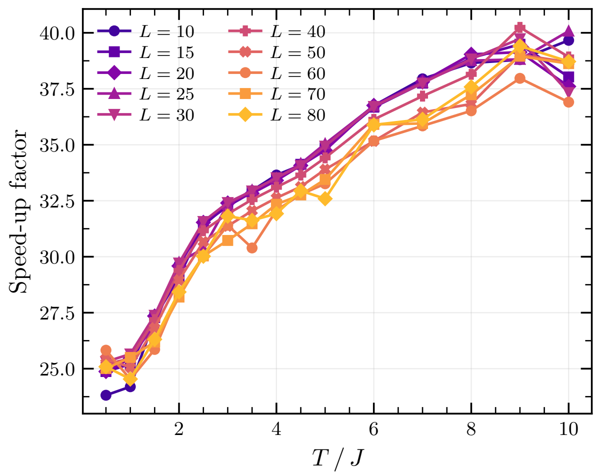
with $D_{ki} = K_{ki} \odot C_{ki} = K_{ki} \odot B_k \odot B_i = K_{ki} \oplus B_k \oplus B_i$. As in the previous section, the flip condition is then:

$$\lfloor \rho \rfloor + n_k \geq 2 \sum_{i\in\partial k} D_{ki}. \tag{15}$$

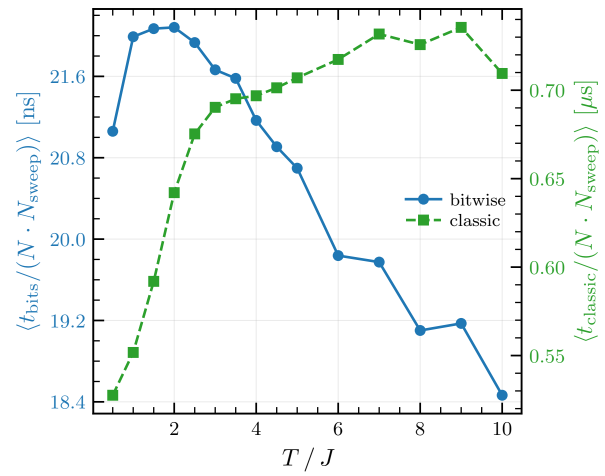
Both the bitwise and the classic implementation are thus similar to this of the Ising model, and we will not present them further in this report. The bitwise algorithm for the spin glass model closely resembles that of the Ising one, it can be found in the appendix, see Algorithm 5.
The bitwise implementation is not validated here, as the bitwise framework was already tested in the previous section, and the main interest of this model lies in bridging the complexity gap between the Ising model and the Hopfield network. We nonetheless verified that the standard and the bitwise implementations produce identical spin configurations when initialized from the same initial state and driven by the same sequence of random numbers.

4.3 Performance test

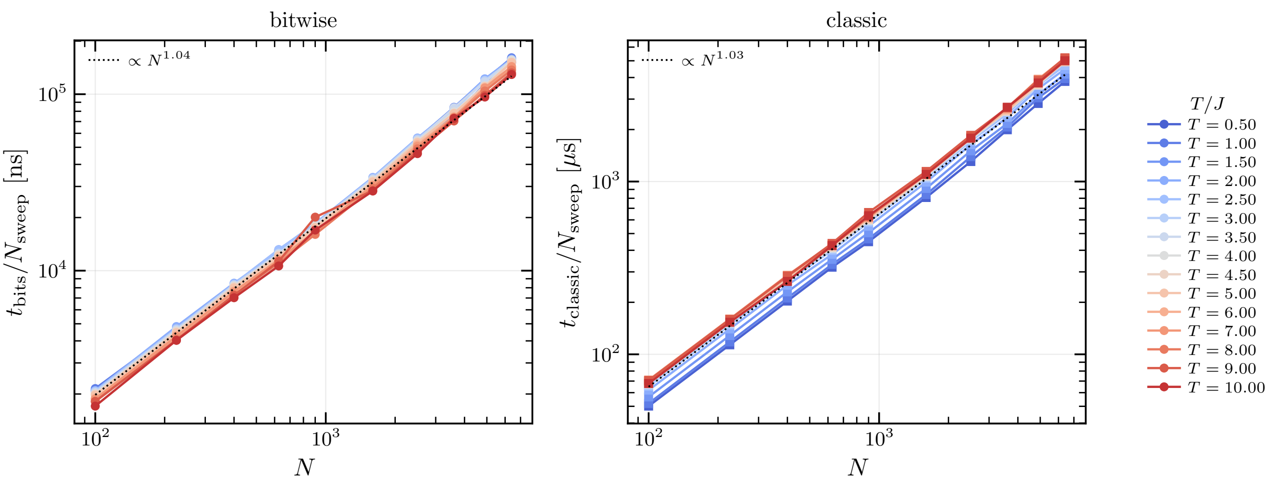
We now test the performance of the implementation, for several lattice sizes and the temperatures, this time with $N_{\mathrm{sweeps}} = 2^{14}$. The results are shown in Figure 10. We notice the same evolution of the normalized time for both implementation as the ones observed for the Ising model. The shape of the speedup factor is similar to this of the Ising model, but its evolution is more monotonic, with what appears to be a saturation for the highest temperatures. The time per sweep also remains proportional to the system size, the complexities of the implementations thus should be similar to those of the Ising model.



(a) Speedup factor evolution.



(b) Normalized time evolution.



(c) Sweep time depending on the lattice size for all temperatures.

Figure 10: Performance analysis of the bitwise and classical Spin Glass Metropolis implementations.

5 The Hopfield model

We here present the Hopfield model, which is of interest in this internship.

5.1 The model

The Hopfield Hamiltonian resemble this of the spin glass model and is as follows:

$$H_{\text{Hopfield}} = -\frac{1}{2}\sum_{i\neq j} w_{ij} S_i S_j, \tag{16}$$

where in this case the weights w_{ij} are not initialized randomly but rather determined by a set of states that one wants to "store" in the network (this term will be explained further on):

$$w_{ij} = \frac{1}{N}\sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu. \tag{17}$$

With $\{\xi^1, \xi^2 \ldots, \xi^p\}$ the set of states to be memorized by the network. Each stored pattern is a state of the system with components $\xi_i^\mu = \pm 1$. This initialization follows the Hebb rule "neurons that fire together wire together" [17]. By setting the weights this way, the energy landscape is shaped with local minimum at each pattern coordinates. Using the definition of m^μ, the energy of the systme can be re-written:

$$w_{ij} = -\frac{N}{2}\sum_{\mu=1}^{p} (m^\mu)^2. \tag{18}$$

By initializing the network not far from one of the stored patters, the dynamics will make it tend to the precise pattern's configuration, given that we are located in the memory phase of the model. In this way, the network is said to have associative memory: it can recover a stored pattern from a partial or noisy version of that pattern. A visual representation of the model is shown in Figure 11.

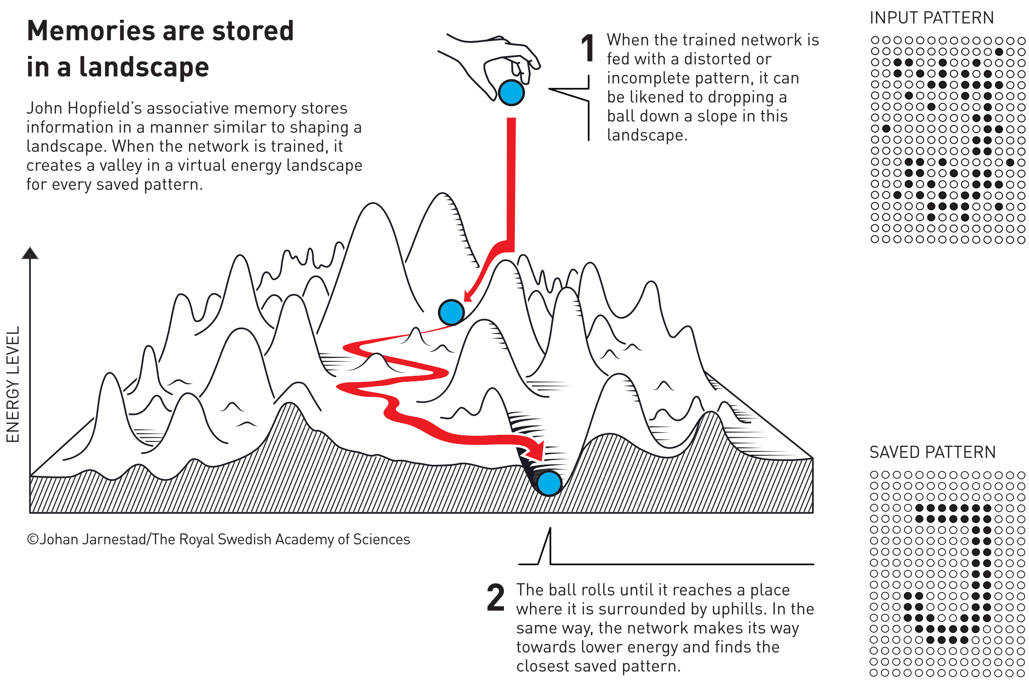


Figure 11: Schematic illustration of the Hopfield model, extracted from [1]. The Hopfield network can recover stored patterns from an altered or partial version of them.

A parameter of particular interest in this model is its storage load, defined as the number of patterns to be stored over the size of the network: $\alpha = p/N$ with p the number of patterns to be stored. The equilibrium properties of the model, depending both on the temperature and on this storage load have been extensively studies in [4–7]. We show in Figure 12 the phase diagram of the Hopfield model on a fully-connected network. In the limit $T \to 0$, the model is known to exhibit a phase transition at $\alpha_c = 0.138$. This storage load value can be regarded as the network's capacity, as beyond this threshold, the model is no longer able to reconstruct the stored data from partial or corrupted data. Above the network's capacity, the energy minima overlap, leading to mixed or spurious (parasite) states that prevent accurate retrieval of stored information.

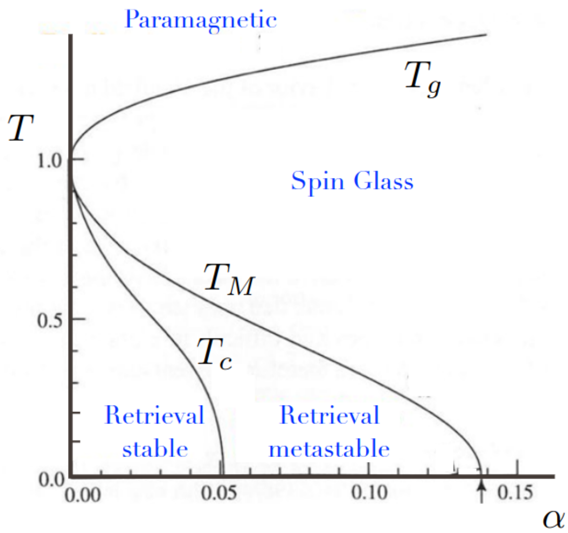


Figure 12: Phase diagram of the Hopfield model, extracted from [18]. the phase transition value at $T = 0$ is signified by the arrow.

In order to study the capacity of the network to recover stored data, we introduce the overlap order parameter:

$$m^\mu = \frac{1}{N} \sum_{i=1}^{N} \xi_i^\mu S_i. \tag{19}$$

This quantity defines how much the system's configuration is aligned to the pattern μ. We often particularly consider the pattern for which the overlap is maximum: $m^* = \max_\mu |m^\mu|$. We take the absolute value of the overlap as the system can also end up in the configuration opposite of the stored pattern due to the $\mathbb{Z}_2$ global spin flip symmetry. All the m^μ are of order $1/\sqrt{N}$ for a state that is uncorrelated with the memories, while, in the memory phase, m^* should be of the order of the unit [7]. From m^* we can define the reconstruction error of the network:

$$\text{error} = \frac{1 - m^*}{2} \tag{20}$$

We show in Figure 5 the evolution of the error with the storage load:

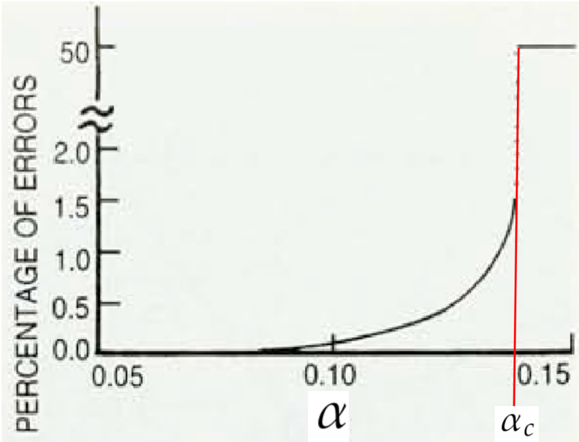


Figure 13: Evolution of the error with the storage load, extracted from [7]. For this diagram, the configuration is initialized near a pattern and we let the system evolve from there. When the storage load exceeds α_c, the configuration becomes random and the error suddenly jumps to 0.5.

5.2 Flip condition

$$\begin{aligned}\Delta E_k &= 2\sum_{i\partial k} w_{ki} S_k S_i \\ &= \frac{2}{N}\sum_{i\partial k}\sum_{\mu=1}^{p} \xi_k^\mu \xi_i^\mu S_k S_i\end{aligned} \tag{21}$$

5.3 Bitwise Implementation

We write $w_{ki} = \frac{1}{N} G_{ki}$ in order to deal with integer-only variables. Thus:

$$\frac{\Delta E_k N}{2} = \sum_{i\partial k} G_{ki} S_i S_k, \qquad \text{with} \quad G_{ki} \in [-p, p] \tag{22}$$

And the flip condition becomes:

$$\lfloor \rho \rfloor \geq \sum_{i\partial k} G_{ki} S_k S_i, \tag{23}$$

with $\rho = -\frac{N}{2\beta}\log r$.

5.3.1 Naive implementation

A first naive implementation would be to unfold G_{ki} into its pattern components in order to deal with `Bits` only objects. The flip condition would then be:

$$\lfloor \rho \rfloor \geq \sum_{i\partial k}\sum_{\mu=1}^{p} \xi_k^\mu \xi_i^\mu S_k S_i \tag{24}$$

As seen in the previous sections, the product $\xi_k^\mu \xi_i^\mu S_k S_i$ can simply be expressed from $X_k^\mu \odot X_i^\mu \odot B_k \odot B_i$, where X_k^μ is the boolean variable associated with ξ_k^μ. This method presents the advantage to not require too much memory allocation to store the couplings G_{ki} as they are not stored before the evolution but computed on the fly in the hot loop. However, this method requires summing over the patterns, thus introducing an additional complexity of $O(p) = O(\alpha N)$ per spin update, which is done $N \cdot N_{\text{sweeps}}$ times.

5.3.2 Optimized implementation

A more efficient way to go would be to stay with the integer factor G_{ki}, which is suitably stored into an `UnsignedInt` in the bitwise implementation. We operate the shift $\tilde{G}_{ki} = p + G_{ki} \in [0, 2p]$, in order to deal with positive only integers, as required by the `UnsignedInt` format. Then:

$$\sum_{i\partial k} G_{ki} S_i S_k = \sum_{i\partial k} (\tilde{G}_{ki} - p)(2C_{ik} - 1) = -\sum_{i\partial k} \tilde{G}_{ki} + 2\sum_{i\partial k} C_{ik}(\tilde{G}_{ki} - p) + n_k p, \tag{25}$$

with $C_{ki} = B_k \odot B_i$. The bitwise flip test is then:

$$\lfloor \rho \rfloor + \sum_{i\partial k} \tilde{G}_{ki} + 2p \sum_{i\partial k} C_{ki} \geq p n_k + 2\sum_{i\partial k} C_{ki} \oplus \tilde{G}_{ki} \tag{26}$$

Where $C_{ki} \oplus \tilde{G}_{ik}$ consists in the XOR operator between C_{ki} and every `Bits` composing the `UnsignedInt` $\tilde{G}_{ki}$:

$$\left(C_{ki} \oplus \tilde{G}_{ki}\right)_l = C_{ki} \oplus \left(\tilde{G}_{ki}\right)_l, \tag{27}$$

where l denotes the l-th `Bits` (associated with the power 2^l). This is strictly equivalent to setting the columns of $\tilde{G}_{ik}$ to 0 where C_{ki} is zero and leave the other ones untouched. Maximum value of R: pN_k
Maximum value of $\sum_{<k,i>} g_{ik}$: $2pN_k$
Maximum value of $2p\sum_{<k,i>} c_{ik}$: $2pN_k$

Maximum value left-hand side: $5pN_k$
Maximum value right-hand side: $pN_k(1 + 2 \cdot 2) = 5pN_k$
The Metropolis condition $r \leq e^{-\beta \Delta E_k}$ is equivalent to:

$$R := \frac{-N}{2\beta} \log r \geq \frac{\Delta E_k N}{2} \tag{28}$$

at max, $\Delta E_k N/2 = PN_k$. So if $R \geq PN_k$, unconditional flip.
Else, after substitution:

Algorithm 3: Bitwise Hopfield Metropolis

Input: Bit-replicated spins $\{B_i\}$, couplings `Couplings`, neighbor lists ∂i, number of patterns P
Output: Updated spin configurations $\{B_i\}$

Preallocated variables: `Bits` c_{ij}, `mask`
`UnsignedInt` Σ_c, Σ_{cg}, Σ_g, LHS, RHS, tmp

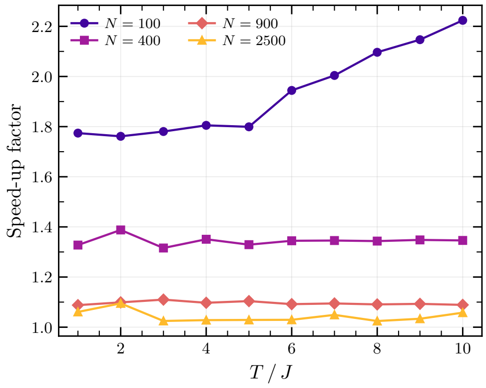
```
for i = 1 to N do
    Σ_g(i) ← 0
    for j ∈ ∂i do
        Σ_g(i) += G_ij
                                                    // Precompute coupling sums
for sweep = 1 to N_sweeps do
    for step = 1 to N do
        Draw a random neuron i
        Draw ⌊ρ⌋ with ρ ~ ℰ(2β)
        if ⌊ρ⌋ ≥ P n_k then
            B_i ← ¬B_i                              // Unconditional flip
        else
            Σ_c ← 0
            Σ_cg ← 0
            for j ∈ ∂i do
                c_ij ← B_i ⊙ B_j                    // Replica-wise agreement
                Σ_c += c_ij
                tmp ← G_ij
                tmp ← tmp ∧ c_ij
                Σ_cg += tmp
            RHS ← P n_i + 2 Σ_cg
            LHS ← Σ_g(i) + r + 2P Σ_c
            mask ← (RHS ≤ LHS)                      // Replica-wise acceptance test
            B_i ← B_i ⊕ mask
```

5.4 Capacity of the Hopfield network

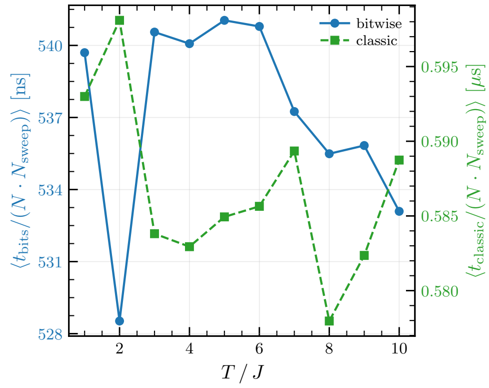
5.5 Performance test

We present in Figure 14 the performance analysis of the code. Several values of α and lattice sizes have been tested for an evolution of 2^{10} sweeps each. Compared to the two previous models, the speedup factor strikingly decreases close to 1 in the case of the dense Hopfield network. The bitwise implementation is no longer more efficient than the reference one. In addition, the speedup factor appears to be mostly uncorrelated with the temperature and the storage capacity of the network.
The main explanation for this drop in efficiency of the bitwise implementation is the topology and the structure of the interactions in the case of the Hopfield model. Indeed, the Hopfield network is most commonly defined as a dense (fully connected) network, meaning that every neuron interacts with every other neuron in the network. This contrasts with the two previous models, in which each spin interacted with only four neighbors; in the Hopfield network, each neuron interacts with all $N-1$ remaining neurons. Thus, if the loop over neighbors at each spin update is not perfectly optimized in the bitwise implementation, the loss in performance can become considerable for dense networks, as observed here. The loss in performance is however much more discreet in the case of the Ising and spin glass models, for which we only sum on the 4 closed neighbors each time.
This hypothesis is confirmed by the decrease of the speedup factor with the lattice size, impacting the number of neighbors to sum over. Also, when testing the code on a sparse Hopfield model, the obtained speedup is very comparable to that of the other models. Conversely, when initialized on a

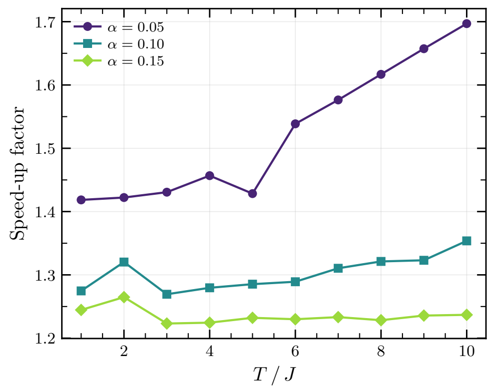
dense network, the Ising model bitwise implementation also yields a significantly reduced speedup factor, confirming again our hypothesis. This is also reassuring, as it shows that the issue does not stem from the generalized use of `UnsignedInt` in this new bitwise implementation, but rather a common lack of optimization in all models, that finds its sources in the coding of the base classes `Bits` and `UnsignedInt`.
The main reason for this lack of efficiency in the neighbors loop is the creation of temporary `Bits` objects over the summations, killing the code performance when done hundreds of thousands of times ($\sim N^2$) per sweep.
Two other related causes can explain the lack of performances for dense networks. The first one is the data structure and the other one is the memory access. The structure has indeed initially been built for sparse networks, with the use of lists of neighbors instead of direct interaction matrices. This implementation is effective when the network is sparse but becomes inefficient in the case of a dense network: instead of storing a fully filled $N \times N$ triangular interaction matrix containing $N\left(N-1\right)/2$ values (taking advantage of the symmetry), we store twice as many values in N lists of $N-1$ components, thus increasing the memory usage, and potentially reducing the performance if the memory overflows the available RAM.



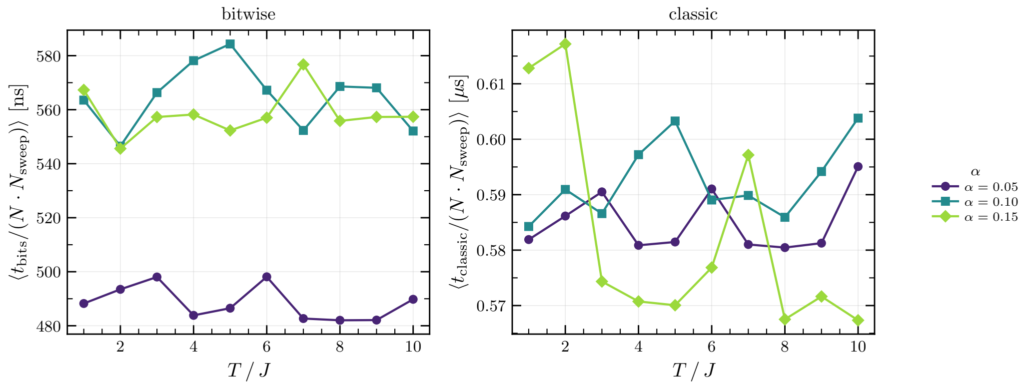
(a) Speedup factor evolution.



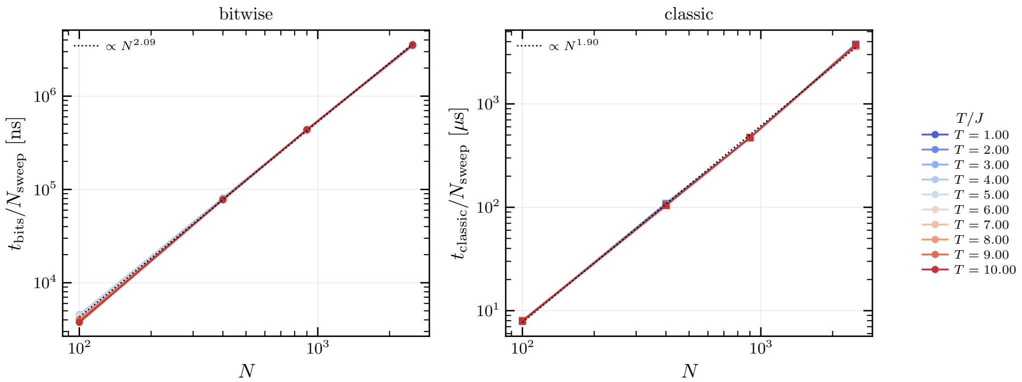
(b) Normalized time evolution.



(c) Speedup factor depending on the temperature for different α values.



(d) Sweep time depending on the temperature for different α values.



(e) Sweep time depending on the lattice size for all temperatures.

Figure 14: Performance analysis of the bitwise and classical Hopfield Metropolis implementations.In Subfigures (a), (b) and (e), the data has been averaged over the different values of α.

6 Conclusion

Appendices

A Naive implementation of the classic Metropolis algorithm

Algorithm 4: Standard Metropolis implementation (naive)

Input: Lattice spins $\{S_i\}$, coupling J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

```
for sweep = 1 to N_sweeps do
    for step = 1 to N do
        Draw k among N
        for r = 1 to n_bits do
            Compute Λ_k^(r)
            // Unconditional flip:  energy is lowered
            if Λ_k^(r) ≤ 0 then
                s_k^(r) ← −s_k^(r)
            else
                // Conditional flip:  draw Poisson threshold
                Draw ⌊ρ⌋ with ρ ~ ℰ(2βJ)
                if ⌊ρ⌋ ≥ Λ_k^(r) then
                    s_k^(r) ← −s_k^(r)
```

B Bitwise implementation of the Metropolis algorithm for the spin glass model

Algorithm 5: Bitwise spin glass Metropolis

Input: Bit-replicated spins $\{B_i\}$, couplings `Couplings`, neighbor lists ∂i, inverse temperature β
Output: Updated spin configurations $\{B_i\}$

Preallocated variables: `Bits xnor, mask`
`UnsignedInt` T_i, `threshold`

```
for sweep = 1 to N_sweeps do
    for step = 1 to N do
        Draw k among N
        Draw ⌊ρ⌋ with ρ ~ ℰ(2β)
        if ⌊ρ⌋ ≥ n_k then
            B_k ← ¬B_k     // escape condition:  unconditional flip across all replicas
        else
            T_k ← 0
            threshold ← 0
            for i ∈ ∂k do
                xnor ← K_ki ⊕ B_k ⊕ B_i                // bitwise XNOR across replicas
                T_k += xnor                   // increment agreement count per replica
            T_k ← 2T_k
            threshold ← ⌊ρ⌋ + n_k
            mask ← (T_k ≤ threshold)                      // per-replica flip condition
            B_k ← B_k ⊕ mask       // bitwise spin flip (XOR operation):  only replicas
                                           // for which mask^(ℓ) = 1 are flipped
```

References

[1] The Royal Swedish Academy of Sciences. Press release: The Nobel Prize in Physics 2024. `https://www.nobelprize.org/prizes/physics/2024/press-release/`, October 2024. Accessed: June 2026.

[2] Shun-ichi Amari. Learning patterns and pattern sequences by self-organizing nets of threshold elements. *IEEE Transactions on Computers*, C-21:1197–1206, 1972.

[3] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.

[4] Daniel J. Amit, Hanoch Gutfreund, and H. Sompolinsky. Spin-glass models of neural networks. *Phys. Rev. A*, 32:1007–1018, Aug 1985.

[5] Daniel J Amit, Hanoch Gutfreund, and H Sompolinsky. Statistical mechanics of neural networks near saturation. *Annals of Physics*, 173(1):30–67, 1987.

[6] Daniel J. Amit. *Modeling Brain Function: The World of Attractor Neural Networks*. Cambridge University Press, Cambridge, 1989.

[7] Haim Sompolinsky. Statistical mechanics of neural networks. *Physics Today*, 41(12):70–80, 1988.

[8] John Hertz, Anders Krogh, and Richard G. Palmer. *Introduction to the Theory of Neural Computation*. Santa Fe Institute Studies in the Sciences of Complexity. Addison-Wesley, Redwood City, CA, 1991.

[9] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[10] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 2023.

[11] H Freund and P Grassberger. Multispin coding for spin glasses. *Journal of Physics A: Mathematical and General*, 21(16):L801, aug 1988.

[12] Matteo Palassini and Sergio Caracciolo. Monte carlo simulation of the three-dimensional ising spin glass, 1999.

[13] Lars Onsager. Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65:117–149, 1944.

[14] K. Binder. Finite size scaling analysis of Ising model block distribution functions. *Zeitschrift für Physik B*, 43:119–140, 1981.

[15] S F Edwards and P W Anderson. Theory of spin glasses. *Journal of Physics F: Metal Physics*, 5(5):965, may 1975.

[16] David Sherrington and Scott Kirkpatrick. Solvable model of a spin-glass. *Phys. Rev. Lett.*, 35:1792–1796, Dec 1975.

[17] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York, 1949.

[18] Marc Mézard. Mean-field message-passing equations in the Hopfield model and its generalizations. *Physical Review E*, 95:022117, 2017.